

AWS Certified Generative AI Developer — Professional

What This Task Is About

Two questions: **where do you run the model**, and **why is deploying an LLM different from deploying a traditional ML model?**

Traditional ML model	-> 50 MB, runs on CPU, loads in seconds
LLM	-> 100+ GB, needs GPU, loads in minutes

That difference is why conventional deployment practice breaks down.

The tiered deployment pattern

Tier 1: Simple queries	-> Lambda (lightweight models)
Tier 2: Medium complexity	-> Bedrock (provisioned throughput)
Tier 3: Complex queries	-> SageMaker endpoints (custom models)
^	
Step Functions router + ElastiCache response cache	

The central idea of Task 2.2: not every query goes to the largest model. Route by complexity and cache what repeats.

Task 1.1 covered deployment options at overview level; this task is the full treatment.

1. Deployment Options

Bedrock On-Demand

- Pay per token, no infrastructure, simplest path
- **Use when:** traffic is uneven, during experimentation, no commitment wanted
- **Limitation:** shared capacity — **throttling is possible at peak**

Bedrock Provisioned Throughput

- Dedicated capacity reserved for you, purchased in **Model Units (MU)**, each providing a guaranteed tokens-per-second rate
- **Sizing:** peak requests/sec × average tokens/request = required TPS → number of Model Units
- **Use when:** steady high volume, predictable latency required, throttling unacceptable
- **Trade-off:** a term commitment (e.g. 1 or 6 months) — you pay even when traffic drops
- Monitor utilisation in **CloudWatch**: under-use wastes money, over-use throttles

SageMaker AI Endpoints

Your own model on your own infrastructure. Choose when the model isn't on Bedrock, you're hosting a fine-tuned model, or you need container-level control and specialised optimisation libraries.

Endpoint type	Use when
Real-time	Always on, low latency, billed per hour
Serverless	Intermittent traffic, cold starts acceptable
Asynchronous	Large payloads, long inference, queue-based
Batch transform	Offline bulk processing

Real-time endpoints provide standardised APIs, SDK integration, IAM authentication, and built-in CloudWatch metrics for throughput, latency and resource utilisation.

Lambda — invocation, not hosting

Lambda does not host the model — it calls it. Configure memory for payload size, timeouts long enough for model responses (seconds, not milliseconds), and retry with exponential backoff.

Bedrock Custom Model Import

fine-tune in SageMaker AI	
->	Bedrock Custom Model Import
->	invoke through the standard InvokeModel API

The benefit the notes emphasise: on-demand access *without* the cost of provisioned throughput. Your custom model, but serverless pricing — no hourly charge for a running SageMaker endpoint.

Exam signal: custom model + cost optimisation → Custom Model Import.

Hybrid solutions

```
Step Functions router
+-- simple  -> Lambda + lightweight model
+-- medium  -> Bedrock provisioned throughput
+-- complex -> SageMaker custom endpoint
```

Task 2.1 routed between *models*; this routes between *hosting platforms*. Same idea, different layer.

Queuing to avoid request denial

```
App -> SQS -> consumer -> Bedrock
```

- Requests are never rejected — they wait in the queue
- The consumer drains at its own rate, so throttling doesn't occur
- Traffic spikes are absorbed
- **Trade-off:** latency increases, so only where an immediate answer isn't required

This is the "async by default" principle from Task 1.1, applied at the deployment layer.

Monitoring and security

- **CloudWatch:** latency, throughput, error rates, resource utilisation — with alarms and dashboards
- **IAM** for who may invoke which model; **VPC** to keep endpoints private; **encryption** for model artifacts and requests
- **AWS PrivateLink** connects a SageMaker endpoint directly to your VPC — **low latency and privacy together**, with traffic never traversing the internet

2. LLM-Specific Deployment Challenges

Size and loading time

Model size:	up to 500 GB
Container health check:	up to 60 minutes
Model download timeout:	up to 60 minutes

Why these timeouts must be raised: the default health check is 5-10 minutes, but downloading 100 GB from S3 and loading it into GPU memory can take 20-40. **With default settings the deployment fails** — SageMaker concludes the container is dead when it was simply still loading. This is a common mistake.

Instance selection

Instance	Suited to
ml.p4d.24xlarge	Large LLMs needing significant compute (named in the notes)
ml.g5.*	Medium LLMs — cost-effective GPU
ml.c5.9xlarge (CPU)	Smaller models such as NER — no GPU required

Not every model needs a GPU. Putting a small task-specific model on GPU wastes money. Choose by model size and throughput requirement.

Model parallelism — when the model exceeds one GPU

A GPU has, say, 80 GB; the model is 200 GB. The model must be **sharded across GPUs**.

Library	What it is
Triton	NVIDIA inference server
FasterTransformer	Optimised for transformer architectures
DeepSpeed	Microsoft — model parallelism and memory optimisation

The notes add the condition: *provided they are compatible with SageMaker AI*.

UltraServers connect multiple EC2 instances via **low-latency, high-bandwidth accelerator interconnects** — for very large AI/ML workloads where even one instance's GPUs are insufficient.

Model loading strategies

Strategy	Effect
Lazy loading	Load on first request rather than at deployment — faster startup, slower first request
Model sharding	Split across GPUs (above)
Quantization	Reduce numeric precision to cut memory

FP32 (4 bytes)	baseline
FP16 (2 bytes)	half the memory
INT8 (1 byte)	a quarter
INT4	less again

Less memory, cheaper instances, faster inference — at some accuracy cost. Same principle as vector quantization in Tasks 1.4 and 1.5, applied to model weights.

Container configuration

- Set **memory limits, GPU allocation and CPU resources** explicitly — choosing the instance is not enough
- **Container image choice** matters: framework compatibility, bundled optimisation libraries, memory management
- SageMaker's pre-built **LMI (Large Model Inference)** containers ship with DeepSpeed and FasterTransformer already included

Token processing — unique to LLMs

Technique	How it works
Static batching	Collect N requests, process together
Continuous batching	Join requests into a running batch as they arrive — more efficient, the GPU never idles

- **Context window management** — longer prompts consume more memory per request
- **Efficient prompt handling** — prompt caching, stripping redundant text

Networking for multi-container deployments

- **Timeout settings** — LLM latency exceeds typical defaults

- Connection pooling and load balancing across containers

Lambda-orchestrated endpoint lifecycle

```
processing triggered
-> Lambda initialises the endpoint
-> downloads model artifacts from S3
-> inference runs
-> work completes -> endpoint shut down
```

Why: a real-time endpoint runs — and bills — continuously. If the workload is two hours a day, twenty-two hours are wasted. Orchestrating it on demand removes that.

Traditional ML vs LLM deployment

	Traditional ML	LLM
Size	MB	Hundreds of GB
Compute	Usually CPU	Usually GPU
Load time	Seconds	Minutes — raise the timeouts
Fits one GPU?	Yes	Often not — needs parallelism
Batching	Simple	Continuous batching
Optimisation	Minor	Quantization often essential

3. Optimised Deployment Approaches

Model cascading

```
query arrives
-> small / cheap model
-> confidence check
    above threshold -> answer, done
    below threshold -> escalate to the larger model
```

- Most queries are simple — roughly 80% can be handled by the small model, leaving 20% to escalate
- **Cost can fall several-fold**
- **Trade-off:** escalated queries cost **two calls**, so latency rises for them
- **Therefore:** cascading pays off only when most queries genuinely are simple. If half escalate, you gain little and add latency.

SageMaker deployment options

Option	Structure
Single model endpoint	One model, one endpoint
Multi-model endpoint	Many models sharing infrastructure, loaded into memory on demand and unloaded when idle
Serial inference pipeline	Containers run in sequence — preprocess → model → postprocess
Multi-container endpoint	Multiple containers that can be invoked individually

All support deployment guardrails, VPC support and network isolation.

Multi-model endpoints suit many models each with low traffic and a similar framework. **Trade-off:** cold start when a model isn't already resident in memory.

Inference components — per-model scaling

The weakness of a plain multi-model endpoint is that everything scales together.

```
Endpoint
+-- Component A (model A) -> own scaling policy, own memory
+-- Component B (model B) -> own scaling policy
+-- Component C (model C) -> own scaling policy
```

Accelerator allocation and memory reservation are controlled per component, so a traffic spike on one model scales only that model.

Caching

Level	Avoids
Response caching	Re-running the model for an identical query
Embedding caching	Re-embedding identical text

```
query -> check cache
hit   -> return immediately (no model call)
miss  -> model -> store the response
```

ElastiCache sits alongside the router in the reference pattern. **Caveat:** LLM responses are often unique — caching pays only where queries genuinely repeat (FAQs, common lookups).

Asynchronous inference

- **SageMaker Asynchronous Endpoints** — request queued, result written to S3, notification via SNS
- **Queue-based architecture** — build it yourself with SQS and SNS
- **Benefits:** scales to zero when idle, handles large payloads, tolerates long inference
- **Use for:** batch reports, document processing, overnight jobs

Model compression

Technique	What it does
Quantization	Lower numeric precision (FP32 → INT8)
Pruning	Removes weights that contribute little
Knowledge distillation	A large "teacher" model trains a small "student" model, which then performs near-teacher level on that specific task

All three trade some quality for size and speed. **How much quality is lost must be measured, not assumed.**

Custom inference containers

When standard containers don't fit — specialised libraries, custom optimisation, unusual model requirements — build your own container and deploy it to SageMaker.

Monitoring and continuous testing

- Track model performance, resource utilisation and cost efficiency together
- The notes' phrase: "**data-driven optimisation decisions**" — measure before optimising
- Examples: GPU utilisation at 20% means the instance is oversized; a 5% cache hit rate means the cache isn't earning its place

Everything combined

```
query
-> cache? -> hit -> return immediately
-> miss
-> cascade: small model
    confident? -> answer
    not        -> larger model (multi-model endpoint)
-> cache the response
```

Quick Reference

Requirement	Choice
Uneven traffic, no commitment	Bedrock On-Demand
Steady volume, no throttling	Bedrock Provisioned Throughput (size in Model Units)
Own or fine-tuned model, full control	SageMaker endpoint
Own model but serverless pricing	Bedrock Custom Model Import
Latency doesn't matter, scale to zero	Asynchronous inference
Many low-traffic models	Multi-model endpoint + inference components
Model larger than one GPU	DeepSpeed / FasterTransformer / Triton
Endpoint fails during deployment	Raise health check and download timeouts (up to 60 min)
Cut cost on a mostly-simple workload	Model cascading
Private, low-latency endpoint access	AWS PrivateLink

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.